

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA



TRABAJO GRUPAL

Curso : [1INF50] Diseño de Software
Título : Sistema de Bibliotecas Universitarias
Número de grupo : 11
Nombres y códigos :

Burga Risco, Ariana - 20226705

Arzapalo Arana, John Manuel - 20202098

Serpa Paredes, Claudia Stephanie - 20212568

Lara Figueroa, Ricardo Valentino - 20221548

Zavala Viscardo, Juan Diego - 20226608

Miranda Mallqui, Luis Miranda - 20223796

Jefe de Práctica :

SEMESTRE 2025-1

TABLA DE CONTENIDO

2	Introducción	3
3	Catálogo de Requerimientos	3
3.1	Requerimientos	3
3.2	Restricciones	8
3.2.1	Organizacionales	8
3.2.2	Técnicas	8
3.2.3	Legales o Normativas	9
4	Arquitectura de la Solución	10
4.1	Vista de Contexto	10
4.2	Vista de Contenedores	11
4.3	Vista de Componentes	13
4.3.1	Componentes del Contenedor <Sistema de Bibliotecarias Universitarias - API Backend >	13
4.3.2	Componentes del Contenedor <Sistema de Bibliotecarias Universitarias - Aplicación Web>	16
4.4	Vista de Código	19
4.4.1	Vista de Código del Componente <API Backend >	19
4.4.2	Vista de Código del Componente de Perfil de Usuario - Aplicación Web	32
5	Plan de Pruebas	39
5.1	Contexto de las Pruebas.....	39
5.1.1	Uso del Sistema	39
5.1.2	Elementos a Probar	39
5.1.3	Alcance de las Pruebas	40
5.1.4	Asunciones	40
5.1.5	Restricciones.....	40
5.2	Comunicación del Plan	40
5.3	Enfoque de Pruebas	40
5.4	Tipos de Pruebas	40
5.5	Riesgos en el Desarrollo de las Pruebas.	40
5.6	Cronograma.	40
5.7	Casos de Pruebas.....	40
5.7.1	Pantalla 1 a probar.	40
5.7.2	Pantalla 2 a probar.	41

1 INTRODUCCIÓN

El Sistema de Bibliotecas Universitarias es una entidad académica fundamental que opera dentro de una o varias instituciones de educación superior. Su rubro de negocio se centra en la gestión y provisión de recursos de información para apoyar las actividades de aprendizaje, docencia e investigación de la comunidad universitaria. Atendiendo a estudiantes, docentes e investigadores, su misión principal es facilitar el acceso al conocimiento y administrar eficientemente el ciclo de vida de su amplio catálogo de materiales bibliográficos.

Para modernizar su operación y mejorar el servicio, la entidad requiere implementar un sistema integrado de gestión de bibliotecas a medida. Este producto software debe centralizar y automatizar sus procesos críticos, incluyendo la administración del catálogo de materiales, el registro de usuarios y el control completo del ciclo de circulación (préstamos y devoluciones). Un componente crucial del sistema es el módulo de sanciones, que debe aplicar de forma automática y precisa las penalizaciones por retraso, daño o pérdida, basándose en el reglamento institucional. Finalmente, el software debe contar con una herramienta de reportes que ofrezca métricas clave sobre el uso de la colección y el comportamiento de los usuarios, facilitando así la toma de decisiones estratégicas.

2 CATÁLOGO DE REQUERIMIENTOS

2.1 REQUERIMIENTOS

ID	Descripción	Actor / Sistema externo	Tipo (Funcional / Calidad)	Prioridad (Alta / Media / Baja)
RF01	El Sistema de Bibliotecas Universitarias deberá permitir el inicio de sesión con credenciales PUCP, mediante correo electrónico o código universitario junto con la contraseña correspondiente.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesisista Bibliotecario Sistema PUCP	Funcional	Alta
RF02	El Sistema de Bibliotecas Universitarias deberá permitir el restablecimiento de la contraseña mediante un redireccionamiento al portal PUCP.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesisista Bibliotecario Sistema PUCP	Funcional	Media

		Sistema de Correo		
RF03	El Sistema de Bibliotecas Universitarias deberá mostrar el perfil del usuario incluyendo sus datos personales, como su código universitario, correo electrónico y nombre completo, y su estado, ya sea activo, inactivo o sancionado.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesisista Bibliotecario	Funcional	Baja
RF04	El Sistema de Bibliotecas Universitarias deberá permitir a los bibliotecarios gestionar los materiales bibliográficos mediante el registro, consulta y modificación de los mismos, incluyendo los datos del material, al menos un contribuyente (autor, editor, ilustrador, traductor), así como al menos un ejemplar con su ubicación y estado.	Bibliotecario	Funcional	Baja
RF05	El Sistema de Bibliotecas Universitarias deberá permitir la búsqueda de materiales bibliográficos mediante una búsqueda simple por el título del material y una búsqueda avanzada, ambas con la opción de filtrar los resultados según criterios seleccionados.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesisista Bibliotecario	Funcional	Media
RF06	El Sistema de Bibliotecas Universitarias deberá permitir ver el detalle de un material bibliográfico, mostrando su información, los ejemplares disponibles por biblioteca, la ubicación de cada ejemplar y el estado de cada uno.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesisista Bibliotecario	Funcional	Alta
RF07	El Sistema de Bibliotecas Universitarias deberá permitir a los bibliotecarios registrar los préstamos de materiales, asociando el ejemplar prestado con el usuario correspondiente mediante su código universitario, y registrando automáticamente la fecha de préstamo y la fecha	Bibliotecario Sistema PUCP Sistema de Correo	Funcional	Alta

	de vencimiento, y actualizando el estado del ejemplar a "prestado". Además, al registrar el préstamo, deberá enviar una constancia del mismo al correo electrónico registrado por el usuario.			
RF08	El Sistema de Bibliotecas Universitarias deberá aplicar límites al total de materiales bibliográficos que pueden estar prestados simultáneamente, permitiendo hasta 7 materiales para estudiantes de pregrado, 15 materiales para estudiantes de posgrado, docentes e investigadores, y 10 materiales para jefes de práctica y tesis. Además, la duración máxima de cada préstamo será de 7 días para estudiantes de pregrado, 15 días para estudiantes de posgrado, jefes de práctica y tesis, y 45 días para docentes e investigadores.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesis	Funcional	Alta
RF09	El Sistema de Bibliotecas Universitarias deberá aplazar la fecha de vencimiento de un préstamo si esta coincide con un día feriado o no laborable, estableciendo una nueva fecha de vencimiento en el siguiente día laborable.		Funcional	Alta
RF10	El Sistema de Bibliotecas Universitarias deberá establecer la fecha de vencimiento del préstamo de un material semestral como el día del fin de ciclo correspondiente.		Funcional	Media
RF11	El Sistema de Bibliotecas Universitarias deberá permitir a los bibliotecarios registrar las devoluciones de materiales bibliográficos, actualizando automáticamente el estado	Bibliotecario Sistema PUCP Sistema de Correo	Funcional	Alta

	del ejemplar a “disponible” y registrando la fecha de devolución. Además, al registrar la devolución, deberá enviar una constancia de la mismo al correo electrónico registrado por el usuario.			
RF12	El Sistema de Bibliotecas Universitarias deberá permitir a los bibliotecarios gestionar las sanciones, incluyendo el registro de sanciones durante la gestión de un préstamo, así como la actualización de su estado y la consulta de las mismas.	Bibliotecario	Funcional	Alta
RF13	El Sistema de Bibliotecas Universitarias deberá mostrar los préstamos realizados por los estudiantes, docentes, investigadores, jefes de práctica y tesis en orden descendente según la fecha de préstamo, permitiendo acceder al detalle de cada préstamo. El detalle deberá incluir la fecha de préstamo, la fecha de vencimiento, la fecha de devolución (si corresponde), el estado del préstamo, la información del material y la sanción asociada (si corresponde), con su fecha de registro, fecha de término y observaciones.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesis	Funcional	Media
RF14	El Sistema de Bibliotecas Universitarias deberá impedir el registro de nuevos préstamos para un estudiante, docente, investigador, jefe de práctica o tesis sancionado mientras la sanción esté activa.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesis	Funcional	Media
RF15	El Sistema de Bibliotecas Universitarias deberá permitir a los bibliotecarios generar reportes en formato PDF, especificando un rango de	Bibliotecario	Funcional	Baja

	fechas, sobre los siguientes temas: los usuarios con más sanciones, los materiales con mayor cantidad de préstamos, y los usuarios según el volumen de préstamos y cumplimiento de devoluciones.			
RNF01	El Sistema de Bibliotecas Universitarias deberá estar estructurado de manera modular, permitiendo que sus componentes puedan ser modificados o ampliados sin afectar otras partes del sistema.		Calidad	Alta
RNF02	El Sistema de Bibliotecas Universitarias deberá ser compatible con diferentes navegadores (Google, Brave, Opera, Zen, Edge).		Calidad	Media
RNF03	El Sistema de Bibliotecas Universitarias deberá mostrar mensajes de error claros y comprensibles, con instrucciones específicas para resolver problemas relacionados con el ingreso de usuario, contraseña o búsqueda de libros.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesista Bibliotecario	Calidad	Alta
RNF04	El Sistema de Bibliotecas Universitarias deberá contar con validaciones para asegurar que los datos ingresados sean correctos y estén completos, esto principalmente en el inicio de sesión.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesista Bibliotecario	Calidad	Alta
RNF05	El Sistema de Bibliotecas Universitarias deberá garantizar que todos los datos sensibles de los usuarios (como la información personal y el historial de préstamos) sean confidenciales, asegurando que solo los usuarios autorizados (administradores, bibliotecarios) tengan acceso		Calidad	Baja

	a ellos, utilizando cifrado de datos tanto en reposo como en tránsito.			
RNF06	El Sistema de Bibliotecas Universitarias deberá responder a las búsquedas realizadas en un tiempo no superior a 5 segundos bajo una carga normal de usuarios.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesisista Bibliotecario	Calidad	Media

2.2 RESTRICCIONES

2.2.1 Organizacionales

- El Sistema de Bibliotecas Universitarias debe respetar la política de manejo de datos sensibles (correo, contraseñas) de la universidad y los estudiantes.
- La capacitación del personal es otro aspecto importante. Se debe hacer un énfasis en el acondicionamiento al manejo del sistema y la resolución de problemas comunes (pérdida de información, errores en el registro de préstamos, conflictos con los estudiantes)
- Uso y acceso solo por miembros de la comunidad PUCP (estudiantes, docentes y personal administrativo) mediante el uso de sus credenciales oficiales.
- El Sistema de Bibliotecas Universitarias debe implementar las políticas de préstamos y sanciones de la universidad (a fecha de este trabajo con última actualización en 2023), como informar al usuario la fecha de devolución de los materiales al momento del préstamo, así como el estado del material al momento de la devolución (conforme, pérdida o robo)

2.2.2 Técnicas

- El Sistema de Bibliotecas Universitarias se implementará siguiendo una arquitectura desacoplada Cliente-Servidor. El API Backend será un servicio independiente que expondrá sus funcionalidades a través de un contrato bien definido.
- El API Backend debe ser desarrollado utilizando el lenguaje de programación Java, específicamente con la versión del JDK 21. Se utilizará el framework Maven para la gestión de dependencias y el ciclo de vida del proyecto. El entorno de ejecución será un servidor de aplicaciones compatible con Java EE/Jakarta EE.
- La Aplicación Web (cliente) debe ser desarrollada utilizando la plataforma .NET de Microsoft, con el lenguaje de programación C# y el framework ASP.NET.
- La comunicación entre la Aplicación Web (C#) y el API Backend (Java) debe realizarse exclusivamente a través del protocolo SOAP (Simple Object Access Protocol) sobre HTTP/HTTPS.

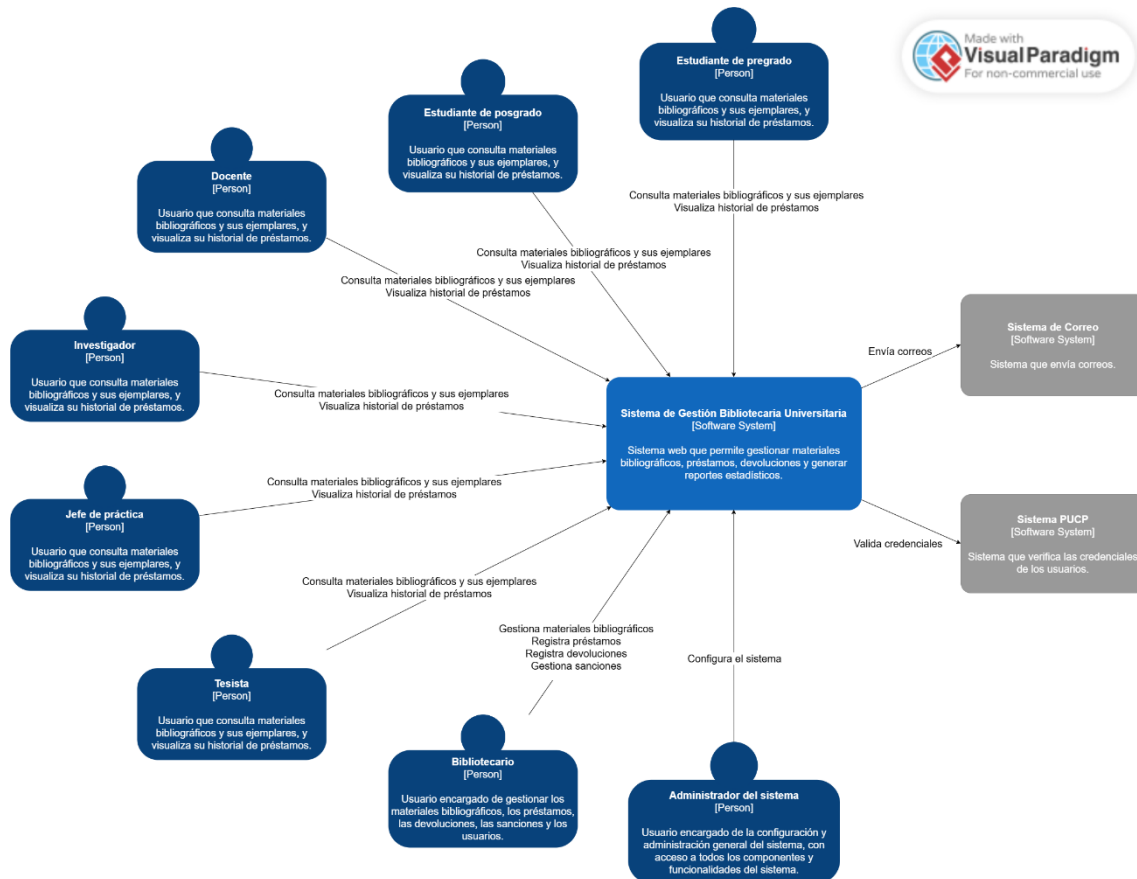
- Toda la información persistente del sistema (usuarios, materiales, transacciones) debe ser almacenada en una base de datos relacional MySQL Server (versión 8.0 o superior). La comunicación entre el API Backend y la base de datos se realizará mediante el protocolo SQL sobre TCP/IP

2.2.3 Legales o Normativas

- El Sistema de Bibliotecas Universitarias deberá cumplir con las pautas de accesibilidad WCAG 2.1, nivel AA, para garantizar que todos los usuarios, incluyendo aquellos con discapacidades, puedan acceder y utilizar el contenido de la plataforma de manera efectiva.
- Cumplimiento con la protección de datos personales (Ley N° 29733), garantizando que esta información no sea compartida sin su consentimiento.

3 ARQUITECTURA DE LA SOLUCIÓN

3.1 VISTA DE CONTEXTO



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

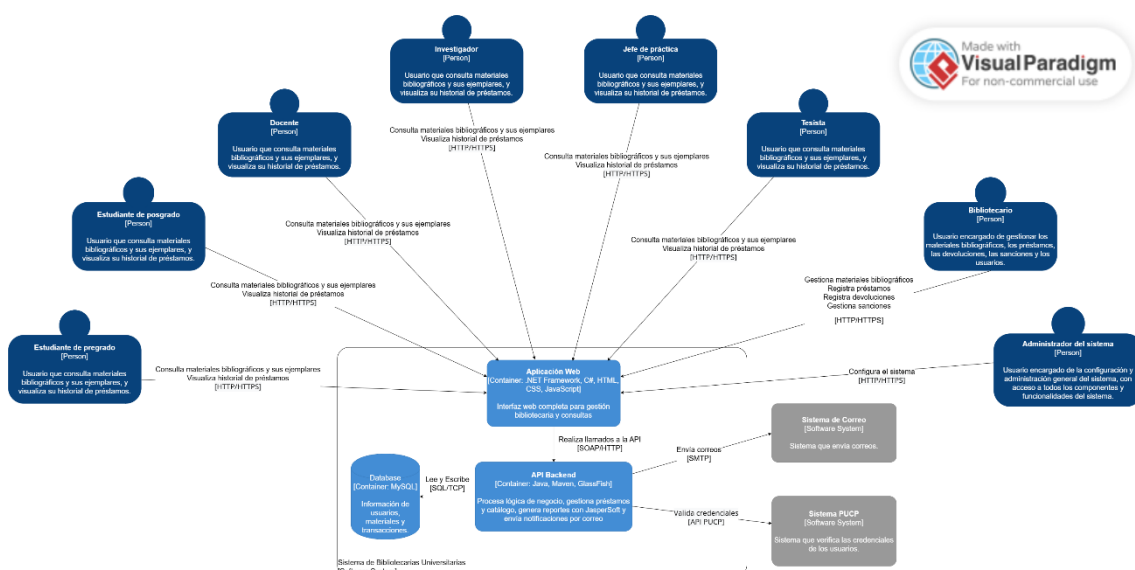
<https://drive.google.com/file/d/1pDzLm4rYnDdivcDVYNY6muiklpmX2I-n/view?usp=sharing>

Actor	Descripción
Estudiante de pregrado	Estudiante que utiliza la plataforma para buscar materiales bibliográficos, ver su información y ejemplares y ver su historial de préstamos.
Estudiante de posgrado	Estudiante que utiliza la plataforma para buscar materiales bibliográficos, ver su información y ejemplares y ver su historial de préstamos.
Docente	Profesor que utiliza la plataforma para buscar materiales bibliográficos, ver su información y ejemplares y ver su historial de préstamos.

Investigador	Investigador que utiliza la plataforma para buscar materiales bibliográficos, ver su información y ejemplares y ver su historial de préstamos.
Jefe de práctica	Jefe de práctica que utiliza la plataforma para buscar materiales bibliográficos, ver su información y ejemplares y ver su historial de préstamos.
Tesista	Tesista que utiliza la plataforma para buscar materiales bibliográficos, ver su información y ejemplares y ver su historial de préstamos.
Bibliotecario	Bibliotecario que utiliza la plataforma para gestionar materiales bibliográficos, registrar y actualizar préstamos y devoluciones, aplicar sanciones, y generar reportes sobre el uso de los recursos, todo mientras mantiene el control sobre los ejemplares.
Administrador del sistema	Usuario responsable de configurar y mantener la plataforma, gestionando los permisos de acceso, asegurando el correcto funcionamiento del sistema y realizando ajustes en la estructura o configuración según sea necesario.

Sistema Externo	Descripción
Sistema de Correo	Sistema que permite enviar correos según las solicitudes de la plataforma.
Sistema PUCP	Sistema que permite validar las credenciales de autenticación de los usuarios.

3.2 VISTA DE CONTENEDORES



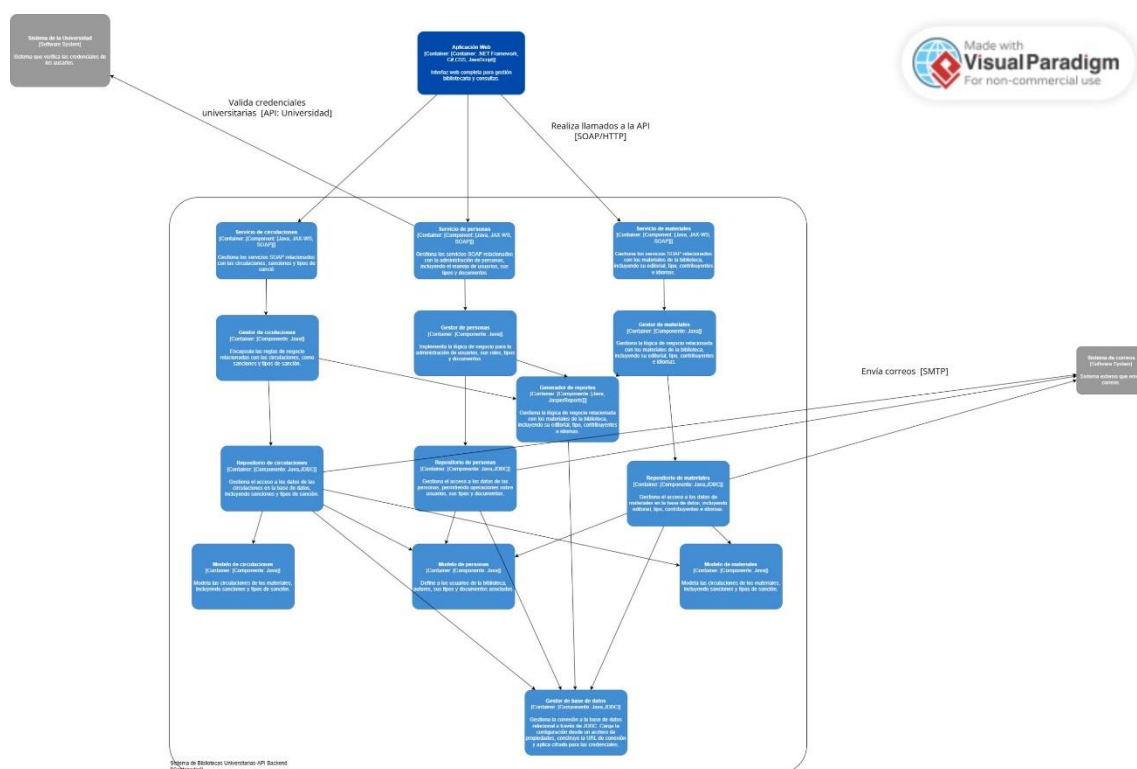
Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

<https://drive.google.com/file/d/1bMxShVcYU3sHKw6pxcaG0BAWvllFoZpF/view?usp=sharing>

Contenedor	Descripción
Aplicación Web	Interfaz web completa desarrollada con .NET Framework, C#, HTML, CSS y JavaScript para la gestión bibliotecaria y consultas. Proporciona funcionalidades para que usuarios, docentes, bibliotecarios y administradores interactúen con el sistema a través de un navegador web.
API Backend	Servicio backend construido con Java, Maven y GlassFish que procesa la lógica de negocio del sistema. Gestiona préstamos y catálogo de materiales, genera reportes con JasperSoft y envía notificaciones por correo electrónico. Actúa como intermediario entre la aplicación web y la base de datos.
Database	Base de datos MySQL que almacena toda la información del sistema incluyendo datos de usuarios (estudiantes, docentes, bibliotecarios, administradores), catálogo de materiales bibliográficos y registro de todas las transacciones.

3.3 VISTA DE COMPONENTES

3.3.1 Componentes del Contenedor <Sistema de Bibliotecarias Universitarias - API Backend >



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

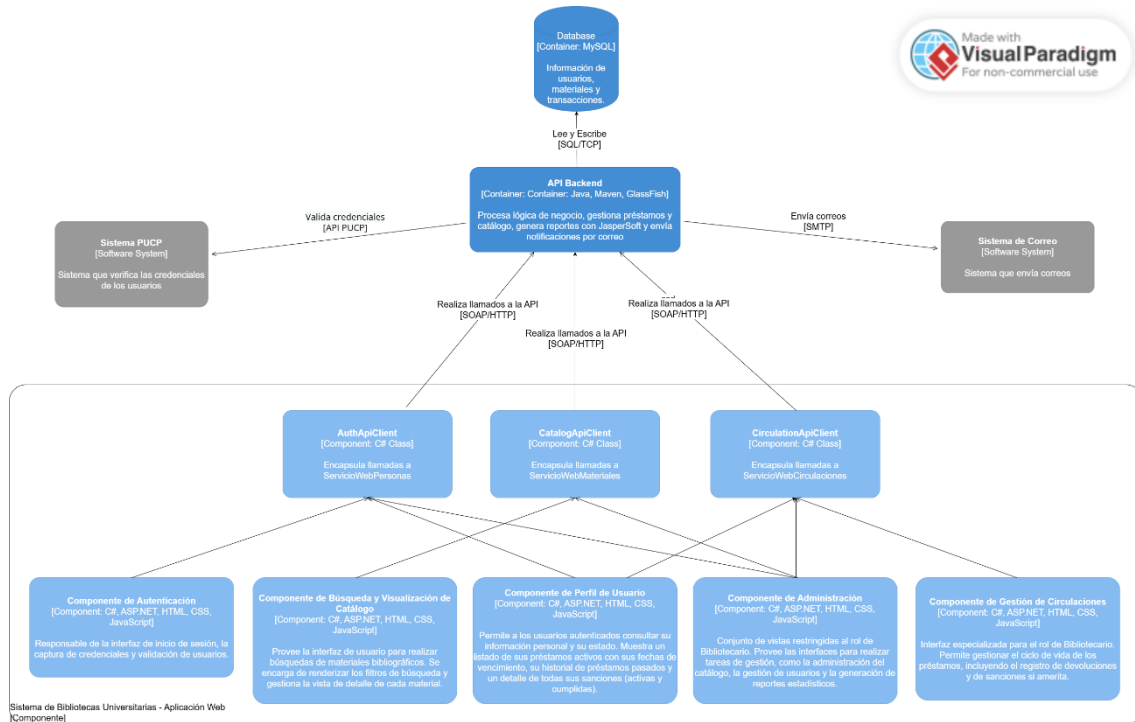
https://drive.google.com/file/d/1ZdGnUDOKd5Ztc-ohftz9slrA8tTpQtWU/view?usp=drive_link

Componente	Descripción	Roles	Req. Funcionales	Req. Calidad
Servicio de Circulaciones	Gestiona los servicios SOAP relacionados con los préstamos, devoluciones y circulación de materiales bibliográficos	Bibliotecario Estudiantes de pregrado Estudiantes de posgrado Docentes Investigadores Jefes de práctica Tesisistas	RF07 RF11 RF13	RNF06
Servicio de Personas	Gestiona los servicios SOAP relacionados con la información de usuarios, autenticación y datos personales	Bibliotecario Estudiantes de pregrado Estudiantes de posgrado Docentes	RF01 RF02 RF03	RNF04 RNF05

		Investigadores Jefes de práctica Tesisistas		
Servicio de Materiales	Gestiona los servicios SOAP relacionados con los materiales bibliográficos, búsquedas y consultas de catálogo	Bibliotecario Estudiantes de pregrado Estudiantes de posgrado Docentes Investigadores Jefes de práctica Tesisistas	RF05 RF06	RNF04 RNF06
Gestor de Circulaciones	Implementa la lógica de negocio para la gestión del flujo de circulación de materiales, incluyendo reglas de préstamo y devolución	Bibliotecario	RF07 RF08 RF09 RF10 RF11 RF12 RF13 RF14	RNF01
Gestor de Personas	Implementa la lógica de negocio para la gestión de usuarios, autenticación, límites por tipo de usuario y sanciones	Bibliotecario Administrador del sistema	RF01 RF02 RF03 RF14	RNF01 RNF05
Gestor de Materiales	Implementa la lógica de negocio para la gestión de materiales bibliográficos, búsquedas, consultas y mantenimiento del catálogo	Bibliotecario Administrador del sistema	RF04 RF05 RF06	RNF01 RNF06
Generador de Reportes	Genera reportes en formato PDF sobre sanciones, préstamos y estadísticas de usuarios según rangos de fechas	Bibliotecario	RF15	RNF06
Repositorio de Circulaciones	Gestiona el acceso a los datos de circulación de materiales, historial de movimientos y transacciones	Administrador del sistema		RNF05

Repositorio de Personas	Gestiona el acceso a los datos de usuarios, credenciales, tipos de usuario y información personal	Administrador del sistema		RNF05
Repositorio de Materiales	Gestiona el acceso a los datos de materiales bibliográficos, ejemplares, autores y metadatos del catálogo	Administrador del sistema		RNF05
Modelo de Circulaciones	Define las entidades de datos para la circulación de materiales, transacciones y movimientos de inventario	Administrador del sistema		RNF01
Modelo de Personas	Define las entidades de datos para usuarios, tipos de usuario, credenciales y información personal	Administrador del sistema		RNF01
Modelo de Materiales	Define las entidades de datos para materiales bibliográficos, ejemplares, autores, contribuyentes y ubicaciones	Administrador del sistema		RNF01
Gestor de Base de Datos	Gestiona la conexión y las operaciones con la base de datos, coordina las transacciones y mantiene la integridad de los datos	Administrador del sistema		RNF01 RNF05 RNF06

3.3.2 Componentes del Contenedor <Sistema de Bibliotecarias Universitarias - Aplicación Web>



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1s86ga3Bczz54JN-PS_2QaVqXfOZtYAMg/view?usp=sharing

Componente	Descripción	Roles	Req. Funcionales	Req. Calidad
AuthApiClient	Encapsula llamadas a ServicioWebPersonas.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesista Bibliotecario	RF01 RF02 RF03 RF04 RF13 RF15	RNF01 RNF03 RNF04 RNF05
CatalogApiClient	Encapsula llamadas a ServicioWebMateriales.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica	RF04 RF05 RF06 RF15	RNF01 RNF03 RNF06

		Tesista Bibliotecario		
CirculationApiClient	Encapsula llamadas a ServicioWebCirculaciones.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesista Bibliotecario	RF03 RF04 RF07 RF11 RF12 RF13 RF15	RNF01 RNF05
Componente de Autenticación	Responsable de la interfaz de inicio de sesión, la captura de credenciales y validación de usuarios.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesista Bibliotecario	RF01 RF02	RNF01 RNF03 RNF04
Componente de Búsqueda y Visualización de Catálogo	Provee la interfaz de usuario para realizar búsquedas de materiales bibliográficos. Se encarga de renderizar los filtros de búsqueda y gestiona la vista de detalle de cada material.	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesista Bibliotecario	RF05 RF06	RNF01 RNF03 RNF06
Componente de Perfil de Usuario	Permite a los usuarios autenticados consultar su información personal y su estado. Muestra un listado de sus préstamos activos con sus fechas de vencimiento, su historial de préstamos pasados y un detalle de todas sus sanciones (activas y cumplidas).	Estudiante de pregrado Estudiante de posgrado Docente Investigador Jefe de práctica Tesista Bibliotecario	RF03 RF13	RNF01 RNF05
Componente de Administración	Conjunto de vistas restringidas al rol de Bibliotecario. Provee las interfaces para realizar tareas de gestión, como la administración del	Bibliotecario	RF04 RF15	RNF01

	catálogo, la gestión de usuarios y la generación de reportes estadísticos.			
Componente de Gestión de Circulaciones	Interfaz especializada para el rol de Bibliotecario. Permite gestionar el ciclo de vida de los préstamos, incluyendo el registro de devoluciones y de sanciones si amerita.	Bibliotecario	RF07 RF11 RF12	RNF01

3.4 VISTA DE CÓDIGO

3.4.1 Vista de Código del Componente de Servicio de Circulaciones - API Backend

Componente de Servicio de Circulaciones



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/13efZJkyL0Bhh8SGQpho6OKf0k7ZUnLxU/view?usp=drive_link

Patrón utilizado: Facade y Data Access Object (DAO)

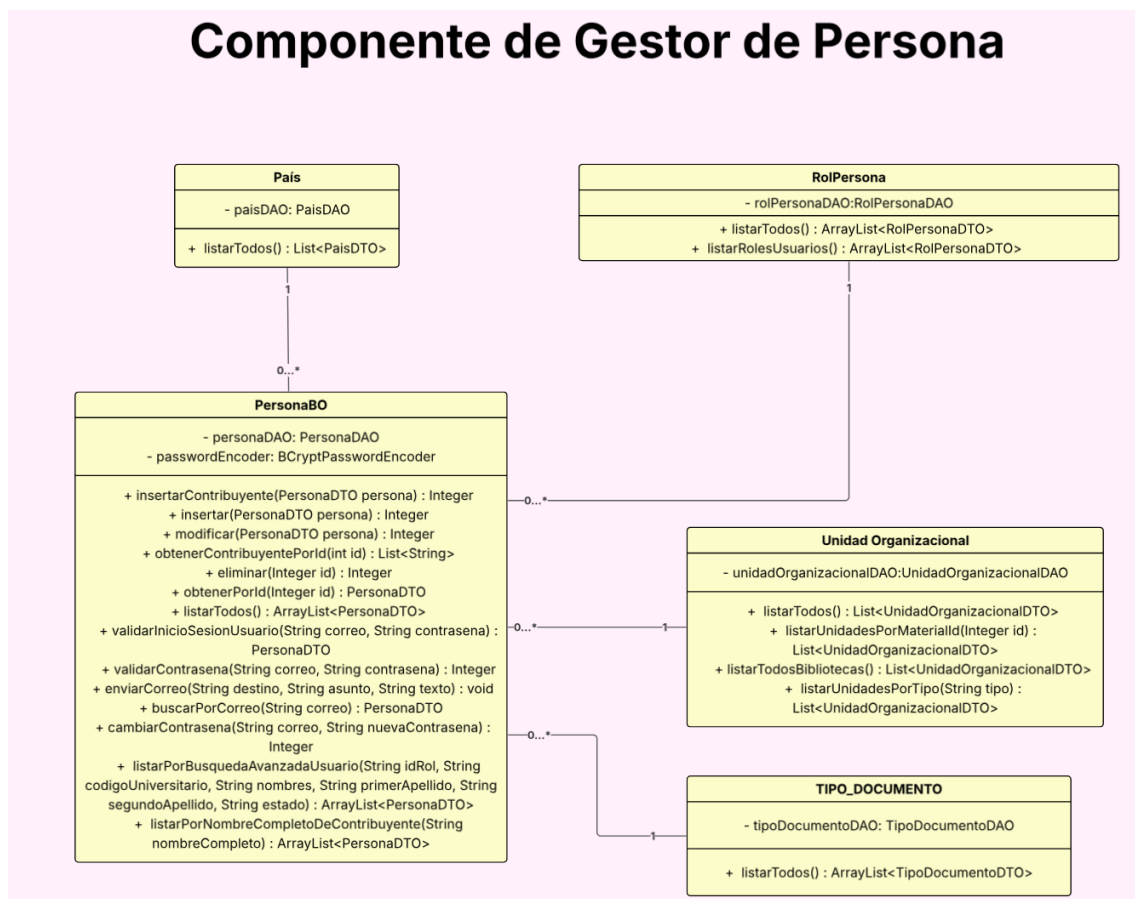
Este componente utiliza dos patrones de diseño principales. Primero, las clases de negocio (Circulacion, Sancion) actúan como una Facade para la lógica de dominio. Encapsulan las reglas de negocio complejas, como la búsqueda avanzada o la generación de reportes, proporcionando una interfaz sencilla para la capa superior (los Servicios Web). Segundo, se emplea el patrón DAO para abstraer y encapsular todos los mecanismos de acceso a la base de datos. Cada clase de negocio delega las operaciones de persistencia (CRUD) a su DAO correspondiente (CirculacionDAO, SancionDAO), separando así la lógica de negocio de la lógica de acceso a datos, lo que mejora la modularidad y facilita el mantenimiento.

Sustento de decisiones de diseño:

- **Separación de Responsabilidades:** El diseño distingue claramente entre la lógica de negocio (en clases como Circulacion) y la persistencia (en las clases DAO). Esto permite modificar la forma en que se accede a la base de datos (por ejemplo, cambiar de JDBC a JPA) sin afectar las reglas de negocio.
- **Cohesión por Dominio:** Las clases están agrupadas por su entidad de dominio (Circulacion, Sancion, TipoSancion). Esto hace que el código sea más organizado, intuitivo y fácil de localizar para futuros desarrolladores.
- **Abstracción de la Persistencia:** La capa de negocio opera con objetos de transferencia de datos (DTOs) y no necesita conocer los detalles de las tablas SQL o las sentencias PreparedStatement. Toda esa complejidad está oculta dentro de la capa DAO, lo que simplifica el código de negocio.

Escenario #1: Administrador busca préstamos vencidos para generar un reporte

Descripción: La capa de Servicios Web recibe una solicitud de la aplicación cliente para generar un reporte de préstamos vencidos. Esta solicitud es delegada a la clase de negocio Circulacion. El método listarPorBusquedaAvanzada de esta clase es invocado con los filtros correspondientes (por ejemplo, un estado "VENCIDO" y un rango de fechas). Internamente, la clase Circulacion utiliza su circulacionDAO para ejecutar la consulta SQL compleja contra la base de datos y obtener una lista de CirculacionDTO. Posteriormente, otro método como generarReportePrestamos toma esta lista de datos, la procesa y formatea (posiblemente usando una librería como JasperReports), y devuelve el reporte como un arreglo de bytes (byte[]), listo para ser enviado de vuelta al cliente.



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1lKGkdjs7VxA1PE39Dp1c474rNS9jZdHw/view?usp=drive_link

Componente: Gestor de Persona (Backend)

Patrón utilizado: Facade y Data Access Object (DAO)

La clase PersonaBO funciona como una Facade para toda la lógica de negocio relacionada con las personas, mientras que las clases como Pais, RolPersona, etc., gestionan sus respectivas entidades. Este componente utiliza el patrón DAO para separar completamente la lógica de negocio de la persistencia de datos. PersonaBO orquesta las operaciones (como validarInicioSesion o listarPorBusquedaAvanzada) y utiliza las clases DAO correspondientes (PersonaDAO, PaisDAO, etc.) para interactuar con la base de datos, encapsulando las consultas SQL y la gestión de conexiones.

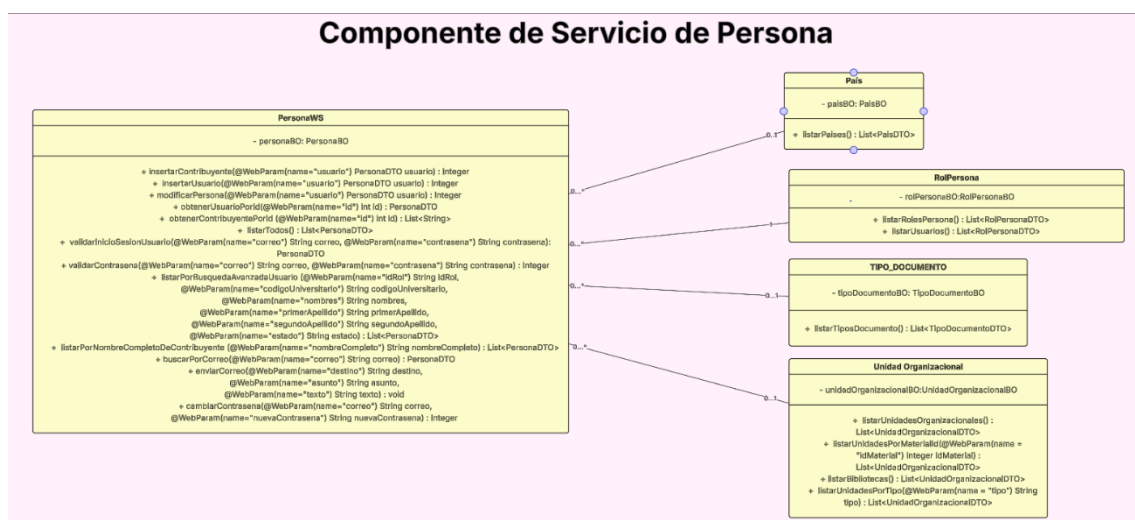
Sustento de decisiones de diseño:

- Separación de Responsabilidades (SoC): La lógica de negocio (BO) está claramente separada de la lógica de persistencia (DAO), lo que permite que cada capa evolucione de forma independiente.
- Centralización de Reglas de Negocio: Todas las reglas de negocio, como la encriptación de contraseñas (BCryptPasswordEncoder) o las validaciones complejas, residen en la capa BO, haciendo el sistema más seguro y coherente.
- Modularidad: La gestión de entidades relacionadas como Pais o RolPersona en sus propias clases de negocio/DAO permite reutilizar estas funcionalidades en otras partes del sistema si fuera necesario.

Escenario #1: Un administrador registra un nuevo usuario

Descripción: La capa de Servicio Web invoca al método insertar(PersonaDTO) de la clase PersonaBO. PersonaBO primero realiza las validaciones de negocio necesarias (ej. verificar si el correo ya existe). Luego, encripta la contraseña del nuevo usuario utilizando BCryptPasswordEncoder. Finalmente, llama al método insertar de su PersonaDAO, que construye y ejecuta la sentencia INSERT en la tabla de usuarios de la base de datos. El resultado de la operación (el ID del nuevo usuario) se propaga de vuelta a través de las capas.

3.4.2 Vista de Código del Componente de Servicio de Personas - API Backend



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1kTHupNbYK1iEGXNozoDt60aTQ71cZi7b/view?usp=drive_link

Patrón utilizado: Facade (Fachada)

La clase PersonaWS actúa como una Facade, exponiendo las funcionalidades de negocio del dominio de personas a través de un servicio web SOAP. Esta clase simplifica la interacción para los clientes externos (como la aplicación web), proporcionando un único punto de entrada (endpoint). Internamente, delega toda la lógica de negocio a la capa de PersonaBO (Business Object), manteniendo así una clara separación entre la capa de exposición del servicio y la capa de lógica de negocio.

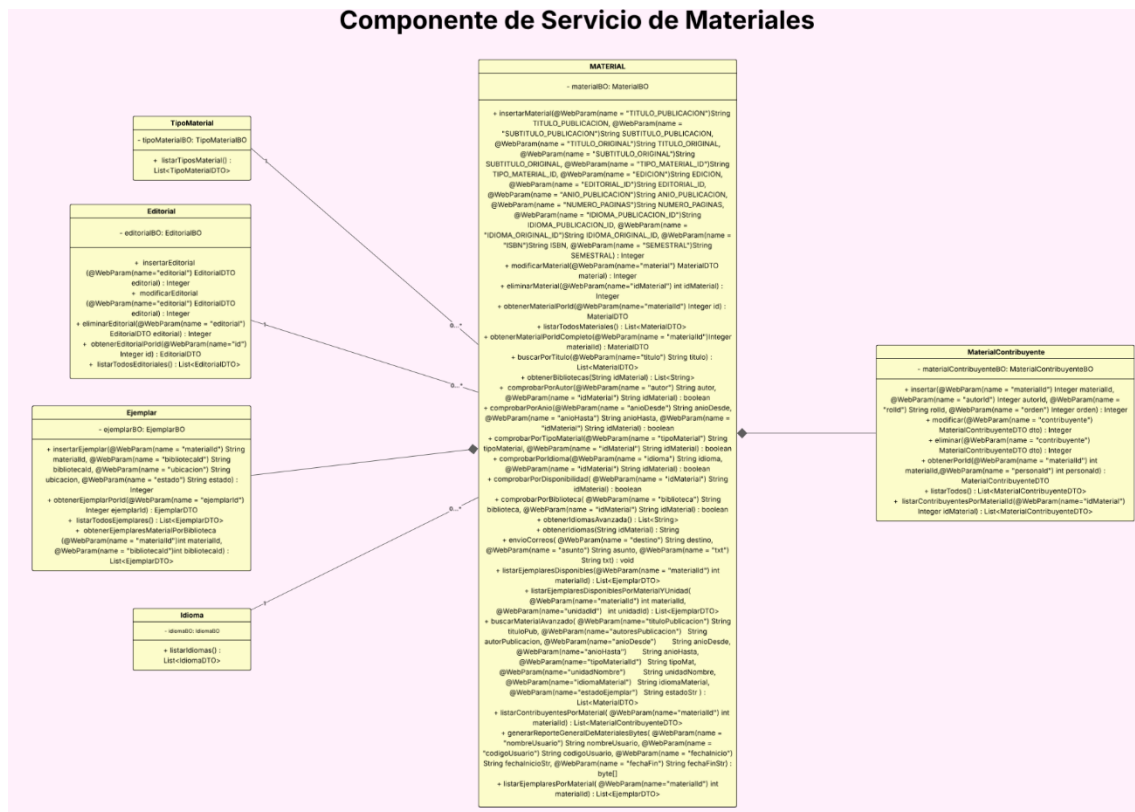
Sustento de decisiones de diseño:

- **Abstracción de la Lógica de Negocio:** La capa de servicio web (PersonaWS) no contiene lógica de negocio; su única responsabilidad es manejar la comunicación SOAP y actuar como un puente hacia la capa de negocio (PersonaBO).
- **Contrato de Servicio Definido:** El uso de @WebMethod establece un contrato estricto (WSDL) que los clientes deben seguir, garantizando una integración robusta y predecible.
- **Cohesión:** Todos los endpoints relacionados con la gestión de personas y su autenticación están agrupados en una sola clase de servicio, lo que mejora la organización del código.

Escenario #1: Cliente web solicita el inicio de sesión de un usuario

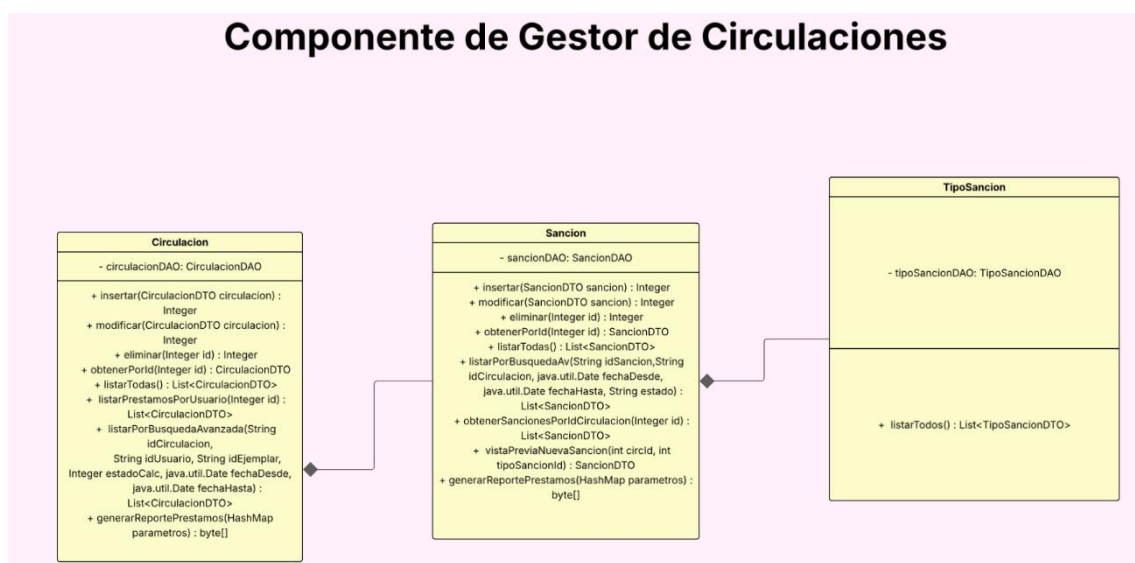
La aplicación cliente (frontend) envía una solicitud SOAP al endpoint validarInicioSesionUsuario de la clase PersonaWS, incluyendo el correo y la contraseña del usuario. PersonaWS recibe esta solicitud y delega la operación a la capa de negocio (PersonaBO), que se encarga de verificar las credenciales contra la base de datos (a través de su DAO) y devuelve el objeto PersonaDTO correspondiente o un nulo. PersonaWS luego serializa esta respuesta y la envía de vuelta al cliente.

3.4.3 Vista de Código del Componente de Servicio de Materiales - API Backend



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

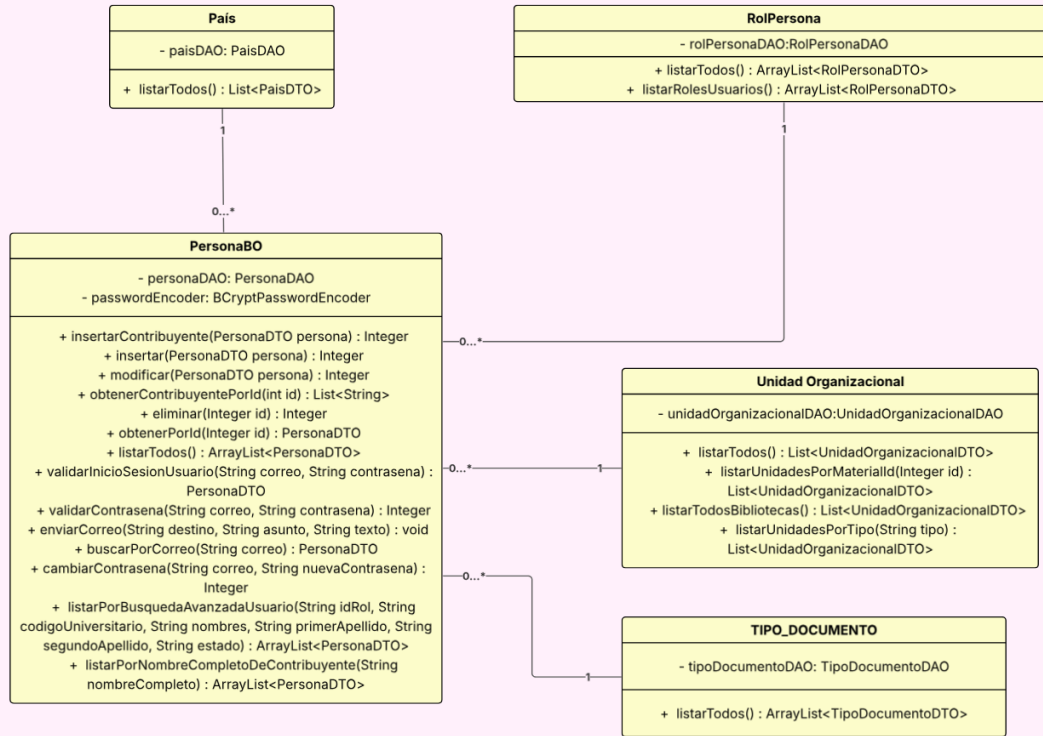
3.4.4 Vista de Código del Componente de Gestor de Circulaciones - API Backend



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

3.4.5 Vista de Código del Componente de Gestor de Personas - API Backend

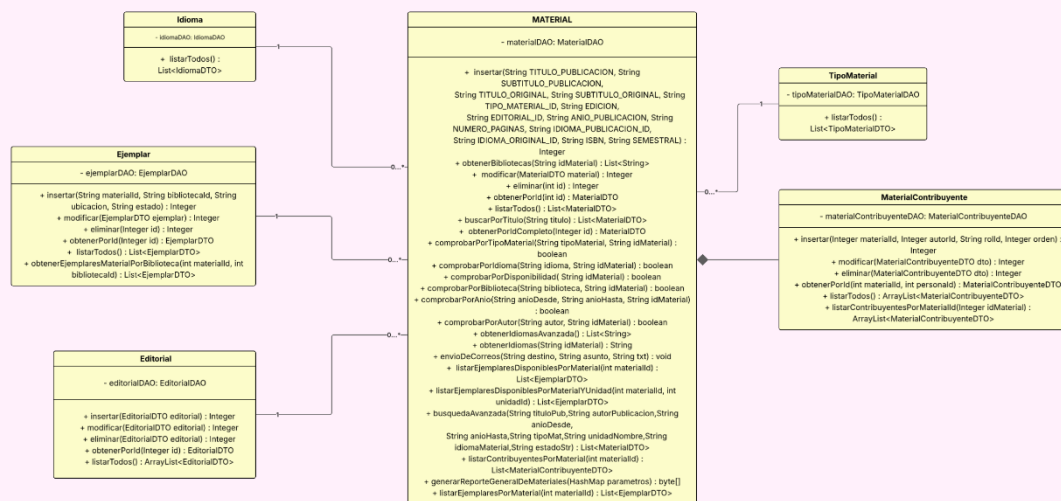
Componente de Gestor de Persona



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

3.4.6 Vista de Código del Componente de Gestor de Materiales - API Backend

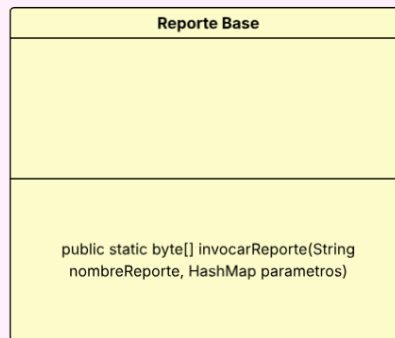
Componente de Gestor de Materiales



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

3.4.7 Vista de Código del Componente de Generador de Reportes - API Backend

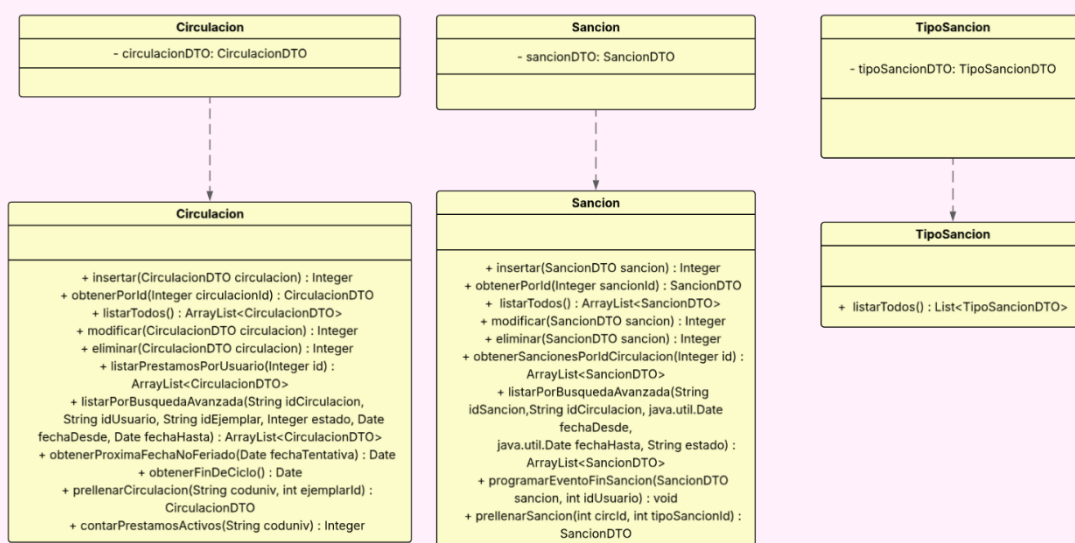
Componente Generador de Reportes



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

3.4.8 Vista de Código del Componente de Repositorio de Circulaciones - API Backend

Componente Repositorio de Circulaciones



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

Patrón utilizado: Data Access Object (DAO)

Este componente implementa el patrón DAO para manejar toda la interacción con la base de datos relacionada con las circulaciones y sanciones. Las clases Circulacion (DAO), Sancion (DAO) y TipoSancion (DAO) encapsulan las operaciones CRUD (Crear, Leer, Actualizar, Borrar) y las consultas SQL específicas. Esta capa abstrae completamente los detalles de la persistencia (JDBC, SQL) de la capa de lógica de negocio, que solo interactúa con estos DAOs a través de métodos claros y objetos de dominio (DTO).

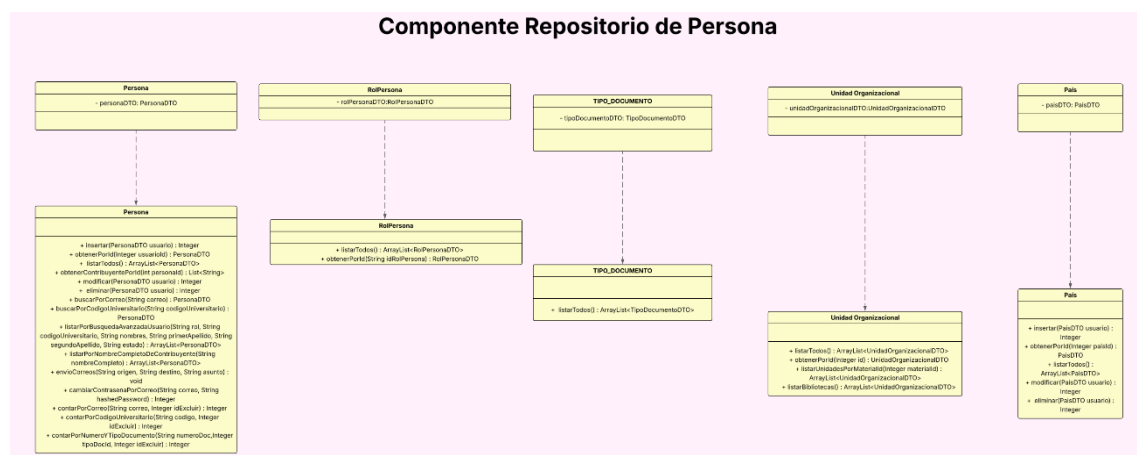
Sustento de decisiones de diseño:

- **Abstracción de la Persistencia:** La lógica de negocio no necesita conocer la estructura de las tablas ni el lenguaje SQL. Solo invoca métodos como insertar(circulacionDTO).
- **Centralización del Acceso a Datos:** Todo el código SQL para una entidad está en un solo lugar, lo que facilita su optimización, depuración y mantenimiento.
- **Intercambiabilidad:** Si se decidiera cambiar el motor de base de datos (de MySQL a PostgreSQL, por ejemplo), solo sería necesario modificar la implementación de esta capa DAO, sin afectar la lógica de negocio.

Escenario #1: Consultar todos los préstamos activos de un usuario

Descripción: La capa de negocio (Gestor de Circulaciones) necesita obtener los préstamos de un usuario. Para ello, invoca el método listarPrestamosPorUsuario(id) de la clase Circulacion (DAO). Esta clase se conecta a la base de datos, ejecuta una sentencia `SELECT * FROM CIRCULACIONES WHERE ID_USUARIO = ? AND ESTADO = 'PRESTADO'`, mapea cada fila del resultado a un CirculacionDTO, y devuelve la lista de objetos a la capa de negocio para su procesamiento.

3.4.9 Vista de Código del Componente de Repositorio de Personas - API Backend



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

Patrón utilizado: Data Access Object (DAO)

Este componente implementa el patrón DAO para gestionar el acceso a los datos de todas las entidades relacionadas con las personas. Clases como Persona, RolPersona, Pais, etc., son en realidad las implementaciones DAO que encapsulan las operaciones de

persistencia. Proporcionan una API simple para que la capa de lógica de negocio (Gestor de Persona) interactúe con la base de datos, ocultando toda la complejidad de las conexiones JDBC, las sentencias SQL y el mapeo de resultados a objetos DTO.

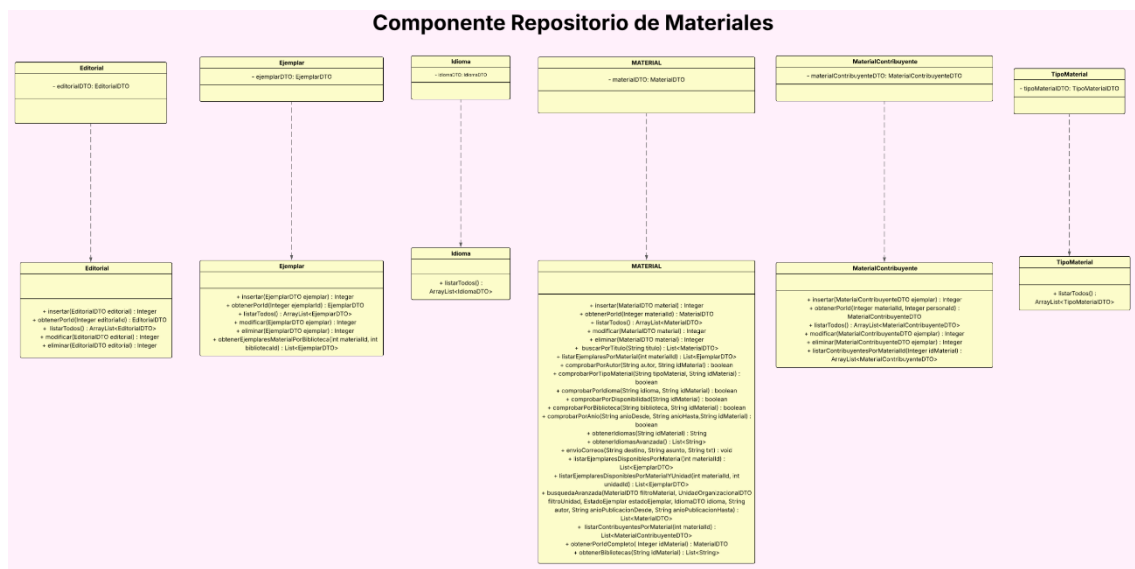
Sustento de decisiones de diseño:

- Separación de Responsabilidades (SoC): La responsabilidad de interactuar con la base de datos está completamente aislada en este componente, separándola de las reglas de negocio.
- Reutilización de Código: Los DAOs para entidades como Pais o TipoDocumento pueden ser reutilizados por diferentes partes de la lógica de negocio que necesiten consultar esta información (ej. al registrar un usuario o un autor).
- Manejo de Transacciones: Esta capa es el lugar ideal para gestionar las transacciones de base de datos (commit/rollback), asegurando la consistencia e integridad de los datos.

Escenario #1: Un administrador realiza una búsqueda avanzada de usuarios

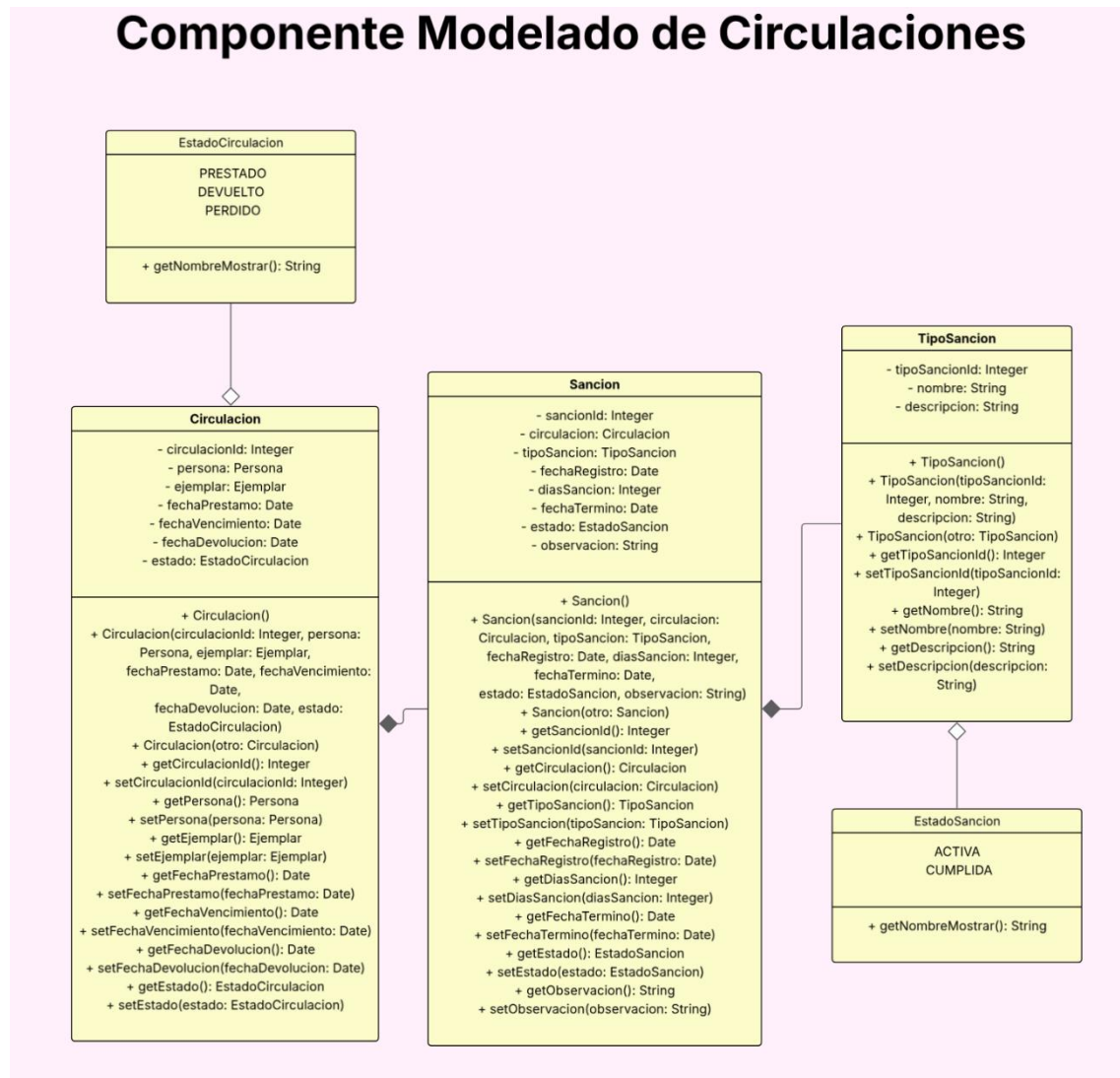
Descripción: La capa de negocio (Gestor de Persona) recibe varios criterios de filtro desde la capa de servicio web. Invoca el método `listarPorBusquedaAvanzada(...)` de la clase Persona (DAO). Esta clase construye dinámicamente una consulta SQL SELECT compleja con múltiples cláusulas WHERE y JOINS a las tablas de roles, unidades organizacionales, etc. Ejecuta la consulta, procesa el `ResultSet`, transforma cada fila en un `PersonaDTO`, y devuelve la lista resultante a la capa de negocio.

3.4.10 Vista de Código del Componente de Repositorio de Materiales - API Backend



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

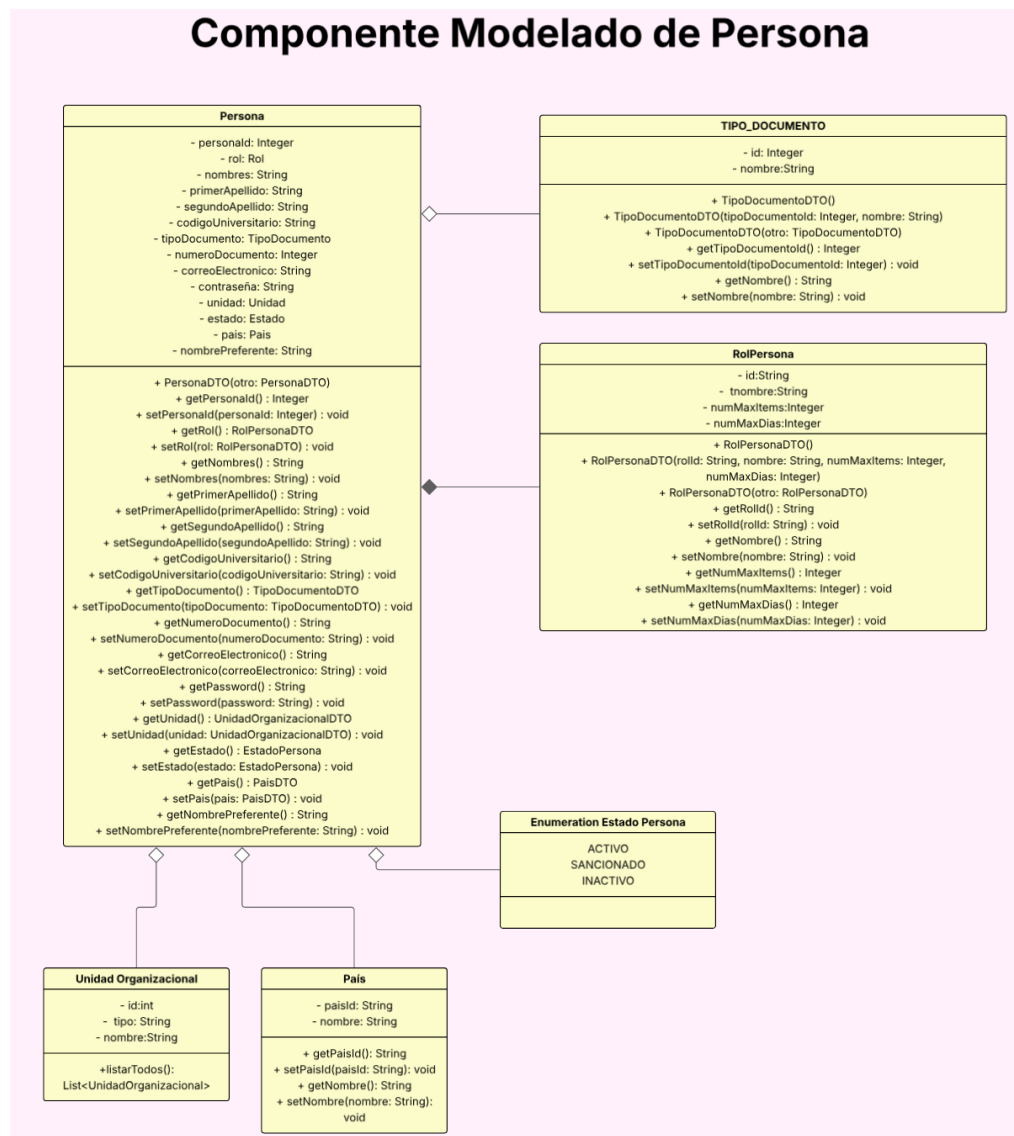
3.4.11 Vista de Código del Componente de Modelo de Circulaciones- API Backend



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1y5TQyTfLmQ7Edc2w1Jh_UPtnw0MFRFTE/view?usp=drive_link

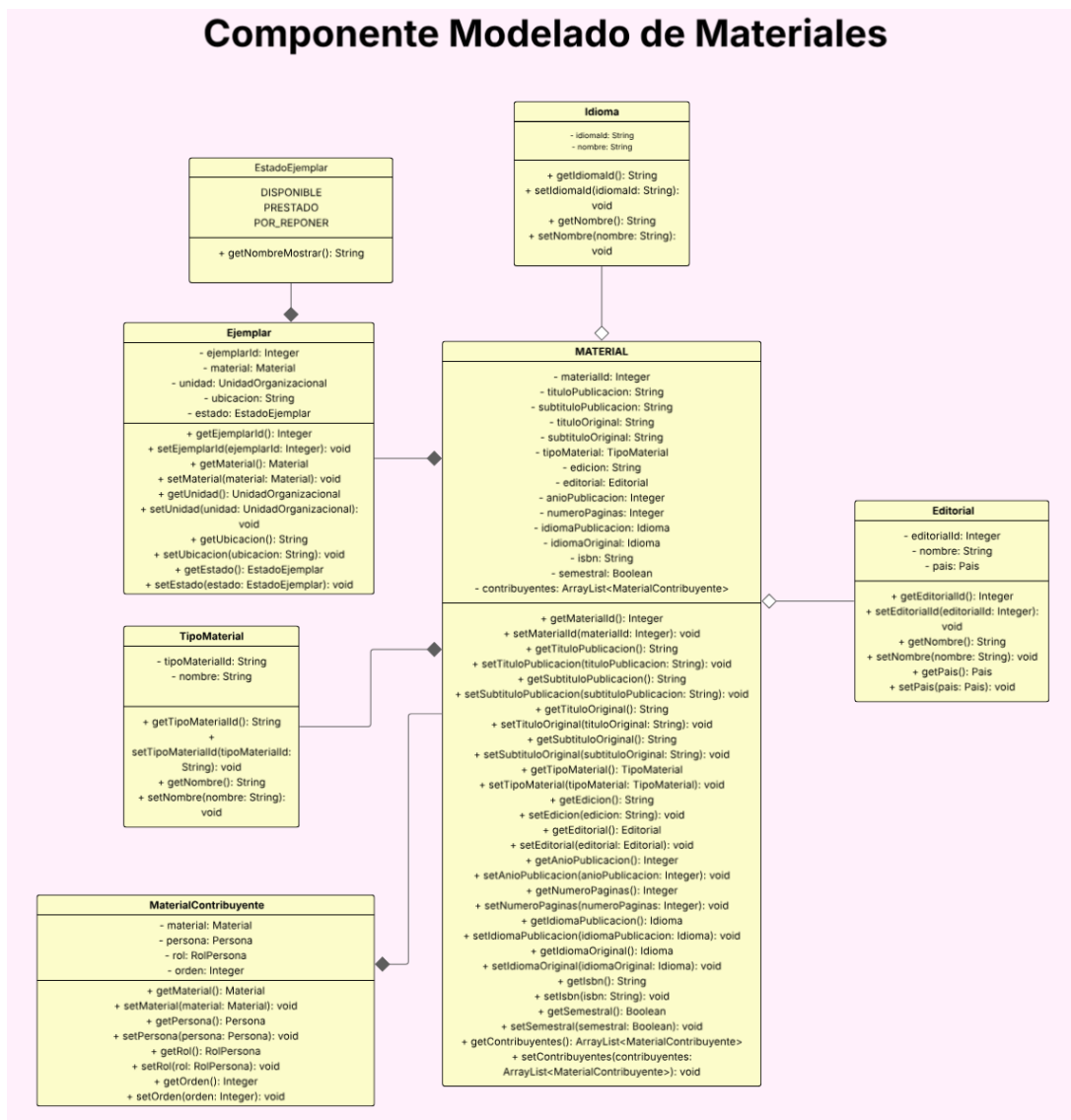
3.4.12 Vista de Código del Componente de Modelo de Personas- API Backend



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1mnBa0jNytMde4HdCuZTlph03uvb02JAl/view?usp=drive_link

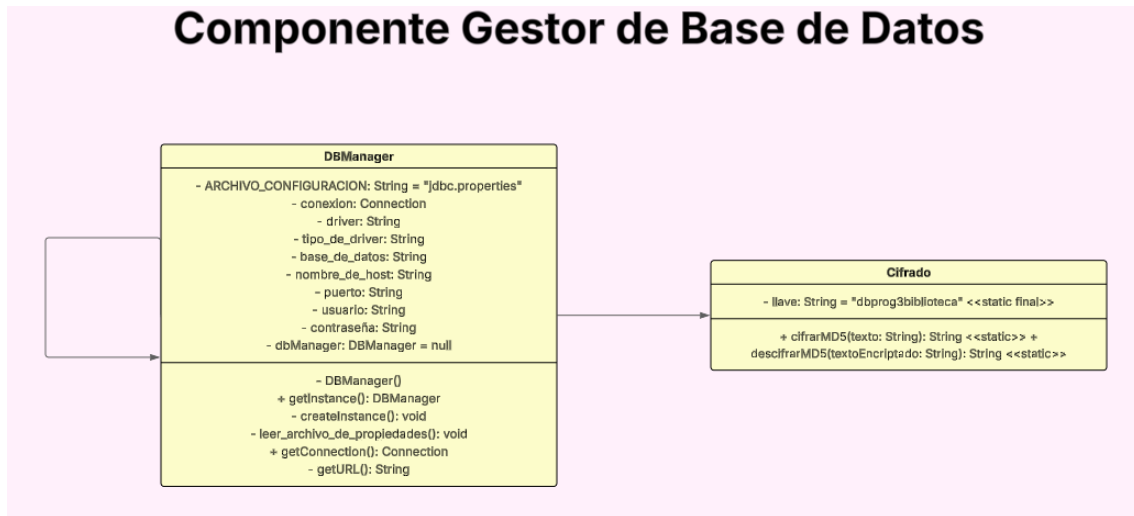
3.4.13 Vista de Código del Componente de Modelo de Materiales - API Backend



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1neLZRSEoNCTETdjdqCxSDrKU5imUC9VH/view?usp=drive_link

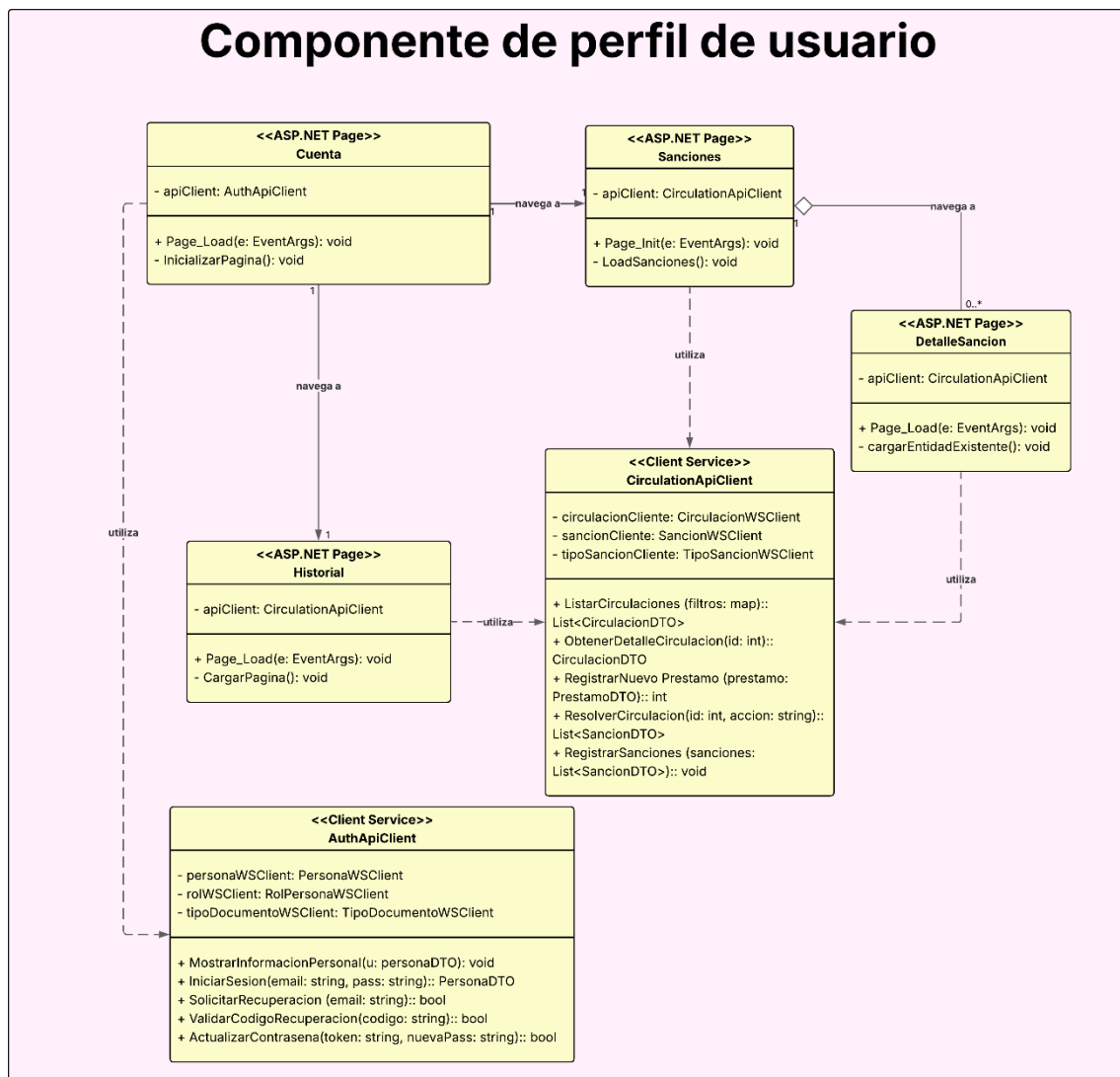
3.4.14 Vista de Código del Componente de Gestor de Base de datos - API Backend



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1KodnoBj5SI4WHPoJ6mPmZ0U8FdWHF8kL/view?usp=drive_link

3.4.15 Vista de Código del Componente de Perfil de Usuario - Aplicación Web



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1LzXInvBEWzkNzJnlmG8Izg_lHEBzwN5X/view?usp=sharing

Patrón utilizado: Facade (Fachada)

El patrón Facade se utiliza en este componente a través de la implementación de las clases **AuthApiClient** y **CirculationApiClient**. Estas clases actúan como una fachada, proporcionando una interfaz simple y unificada para un conjunto más complejo de subsistemas del backend (los **PersonaWS**, **CirculacionWS**, y **SancionWS**). Las páginas de la interfaz de usuario (**Cuenta**, **Historial**, **Sanciones**) no necesitan conocer los detalles de la comunicación SOAP ni interactuar con múltiples servicios del backend; simplemente realizan llamadas a métodos de alto nivel en la fachada, como **ObtenerDatosPersonales()** o **ObtenerHistorialPrestamos()**. Esto reduce el acoplamiento entre la interfaz de usuario y el backend, simplificando el código de las vistas y centralizando la lógica de acceso a datos en un solo lugar.

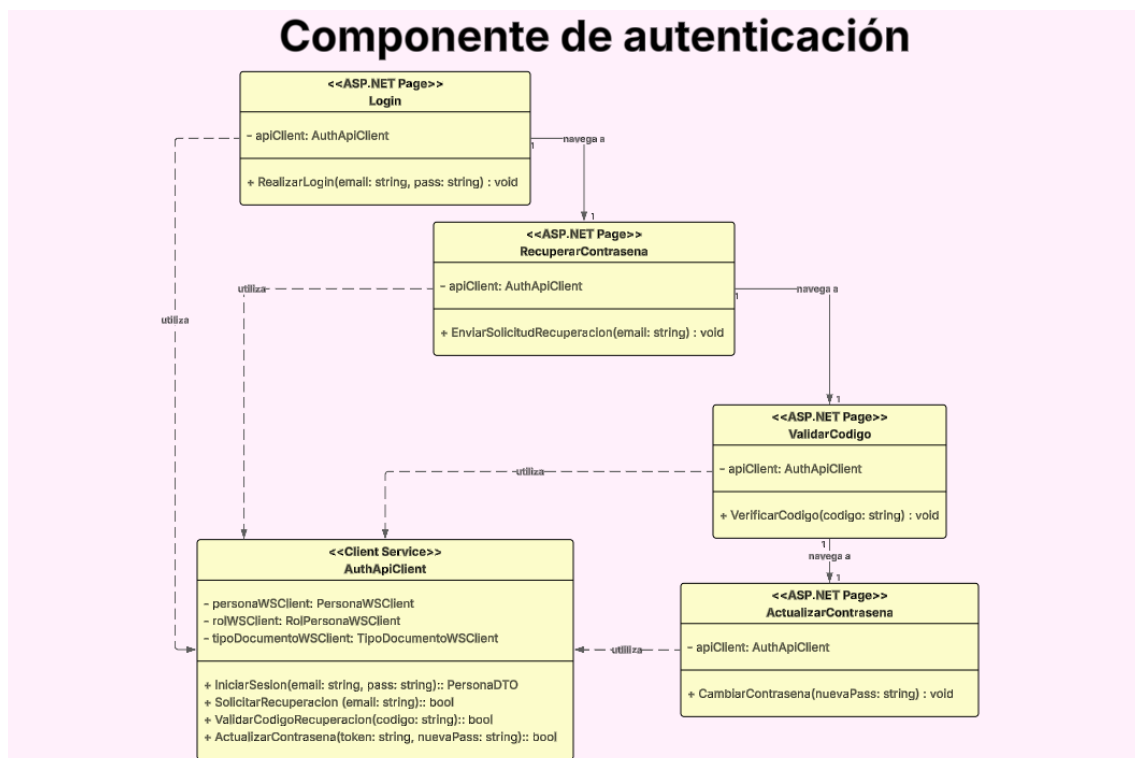
Sustento de decisiones de diseño:

- **Centralización de la Lógica de Cliente:** Se crearon las clases ApiClient para centralizar toda la lógica de comunicación con el backend relacionada con el perfil del usuario. Esto evita la duplicación de código en las diferentes páginas y facilita el mantenimiento.
- **Desacoplamiento de la Interfaz de Usuario:** Las páginas (.aspx.cs) no dependen directamente de la tecnología WSCClient. Si el protocolo de comunicación cambiara de SOAP a REST, solo se necesitaría modificar los ApiClients, sin afectar las clases de la interfaz de usuario.
- **Abstracción de Responsabilidades:** Cada página (Cuenta, Historial, Sanciones) tiene una responsabilidad clara y enfocada en la presentación de datos. Delega toda la tarea de obtención y manejo de datos a la capa de ApiClient, siguiendo el principio de responsabilidad única.

Escenario 1: Usuario consulta su historial de préstamos y sanciones

Descripción: Cuando un usuario autenticado entra a su perfil, la página Cuenta se carga e invoca al AuthApiClient para obtener y mostrar la información personal. Simultáneamente, las secciones de Historial y Sanciones (que pueden ser controles de usuario o páginas separadas) utilizan el CirculationApiClient para solicitar el listado de préstamos pasados y las sanciones activas/cumplidas. El ApiClient se encarga de realizar las llamadas SOAP correspondientes al backend, consolida la información y la devuelve a las vistas para que estas la rendericen y presenten al usuario de forma clara.

3.4.16 Vista de Código del Componente de Autenticación - Aplicación Web



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

<https://drive.google.com/file/d/17Pf5RPNC8v9W-Cl2ywnKWGz0hUfVRiWV/view?usp=sharing>

Patrón utilizado: Facade (Fachada)

En este componente, el patrón Facade se materializa en la clase AuthApiClient. Esta clase proporciona una interfaz simplificada para los procesos de seguridad del sistema, como el inicio de sesión y la recuperación de contraseña. Las páginas de la UI (PaginaLogin, PaginaRecuperarContraseña, etc.) no interactúan con la complejidad de los servicios web de PersonaWS o RolPersonaWS. En su lugar, invocan métodos directos y con una intención clara en la fachada, como IniciarSesion(email, pass). El AuthApiClient encapsula la lógica de construir la petición SOAP, comunicarse con el backend y procesar la respuesta, protegiendo al resto de la aplicación de estos detalles de implementación.

Sustento de decisiones de diseño:

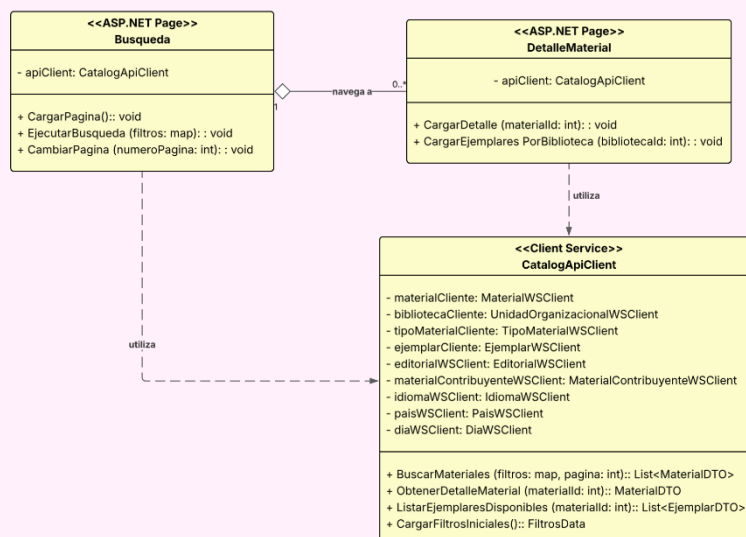
- **Encapsulamiento de la Seguridad:** Toda la lógica de cliente relacionada con la autenticación y la gestión de contraseñas está contenida en el AuthApiClient. Esto crea un único punto de control y facilita las auditorías de seguridad.
- **Abstracción de Flujos Complejos:** El proceso de recuperación de contraseña implica varios pasos (solicitar, validar, actualizar). La fachada lo simplifica para la UI, que solo necesita llamar a un método por cada paso, sin tener que gestionar el estado intermedio del flujo.
- **Mejora de la Testeabilidad:** Al aislar la lógica de comunicación en una clase ApiClient, es más fácil realizar pruebas unitarias sobre las páginas de la UI utilizando un "mock" o simulacro del ApiClient, sin necesidad de tener una conexión real con el backend.

Escenario #1: Usuario inicia sesión en el sistema

Descripción: El usuario ingresa su correo electrónico y contraseña en la PaginaLogin y presiona el botón de "Ingresar". El código de la página captura estas credenciales y las pasa al método IniciarSesion del AuthApiClient. Este cliente construye y envía una solicitud SOAP al ServicioWebPersonas en el backend. El backend valida las credenciales (potencialmente consultando al Sistema PUCP externo). Si la validación es exitosa, el backend devuelve los datos del usuario. El AuthApiClient recibe esta respuesta, la procesa y la devuelve a la PaginaLogin, que entonces crea la sesión local para el usuario y lo redirige a la página principal de la aplicación.

3.4.17 Vista de Código del Componente de Búsqueda y Visualización de Catálogo - Aplicación Web

Componente de búsqueda y visualización de catálogo



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1nLeQR5dpqVHQ2zQM02_WpzpCQxwW24TW/view?usp=sharing

Patrón utilizado: Facade (Fachada)

El `CatalogApiClient` actúa como una fachada que simplifica la interacción con el subsistema de catálogo del backend. La interfaz de usuario, compuesta por la `PaginaBusquedaCatalogo` y la `PaginaDetalleMaterial`, necesita obtener datos de múltiples entidades (materiales, autores, editoriales, ejemplares, etc.). En lugar de que la UI gestione múltiples `WSClients`, interactúa exclusivamente con el `CatalogApiClient`. Esta fachada expone métodos de negocio como `BuscarMateriales(filtros)` u `ObtenerDetalleMaterial(id)`, ocultando la complejidad de las consultas subyacentes y las posibles múltiples llamadas al backend que se necesiten para construir una vista completa de los datos.

Sustento de decisiones de diseño:

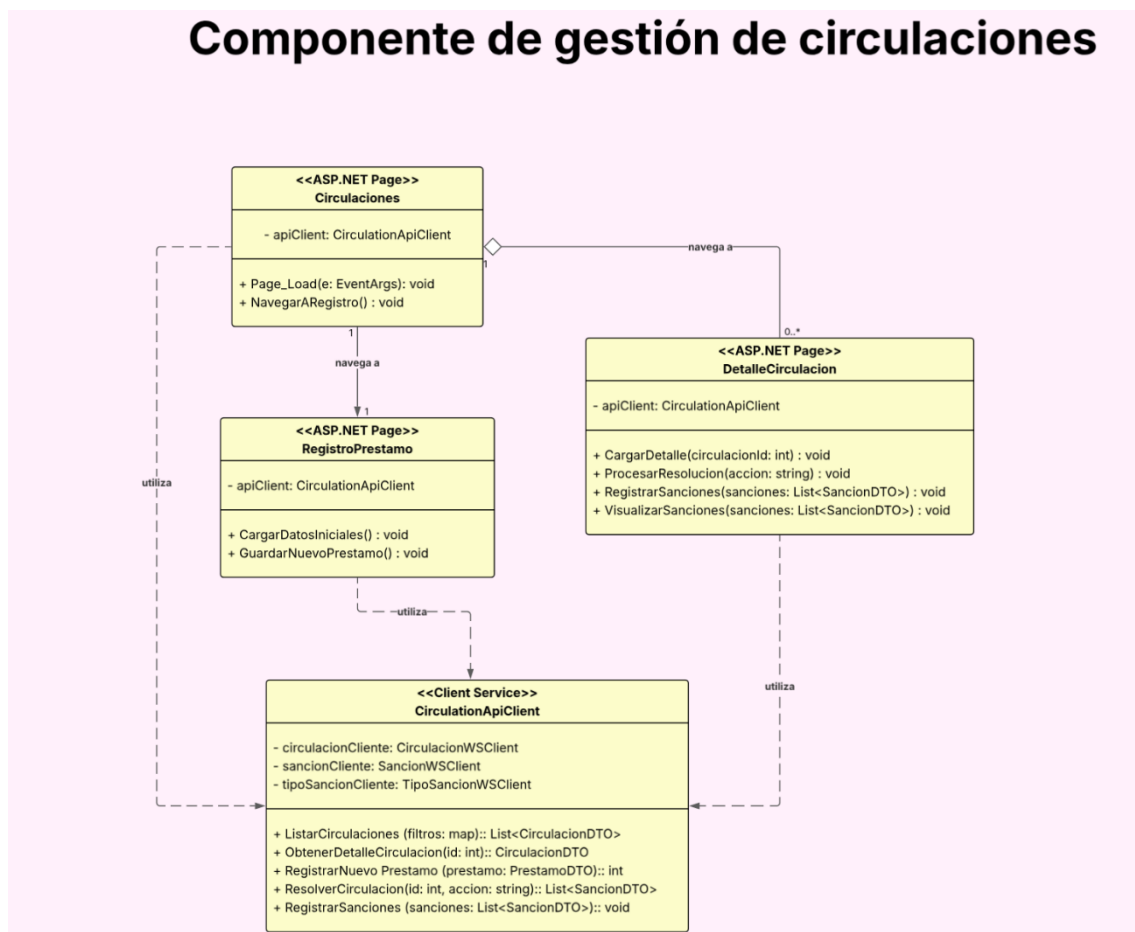
- **Agregación de Datos del Lado del Cliente:** El `CatalogApiClient` puede actuar como un agregador, combinando información de diferentes llamadas al backend (ej. obtener datos del material y luego obtener sus ejemplares) antes de devolver un objeto consolidado a la UI.
- **Abstracción de Consultas Complejas:** La UI no necesita saber cómo construir los complejos objetos de filtro que el backend pueda requerir. Simplemente pasa un objeto de filtro simple al `ApiClient`, y este se encarga de traducirlo al formato que el servicio SOAP espera.

- **Mantenibilidad de la Lógica de Búsqueda:** Si la lógica de búsqueda o los servicios del backend cambian, solo se necesita actualizar el CatalogApiClient. Las páginas de la interfaz de usuario permanecen sin cambios, lo que reduce significativamente el esfuerzo de mantenimiento.

Escenario #1: Usuario busca un libro y visualiza su detalle

Descripción: Un usuario ingresa "Diseño de Software" en la barra de búsqueda de la PaginaBusquedaCatalogo y aplica un filtro por "Biblioteca de Ingeniería". La página invoca al método BuscarMateriales del CatalogApiClient, pasándole los criterios. El ApiClient envía la solicitud SOAP al backend, que devuelve una lista de materiales que coinciden. La página renderiza esta lista. A continuación, el usuario hace clic en un resultado. Esto dispara una navegación a la PaginaDetalleMaterial, pasándole el ID del material. En su Page_Load, esta nueva página utiliza el CatalogApiClient para llamar al método ObtenerDetalleMaterial(id), obteniendo toda la información detallada y la disponibilidad de ejemplares, la cual es finalmente mostrada al usuario.

3.4.18 Vista de Código del Componente de Gestión de Circulaciones - Aplicación Web



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

<https://drive.google.com/file/d/15dcfWkstGDvPm8EmgMA7DXp0RU99DcN/view?usp=sharing>

Patrón utilizado: Facade (Fachada)

El *CirculationApiClient* implementa el patrón Facade para actuar como el único punto de entrada para todas las operaciones de circulación que el personal de la biblioteca realiza desde la interfaz web. Las páginas, como *Circulaciones* o *DetalleCirculacion*, no interactúan directamente con la complejidad de los servicios web de circulación, sanciones o materiales. En su lugar, invocan métodos de negocio claros en la fachada, como *ResolverCirculacion(id, accion)*. Esta fachada orquesta las llamadas necesarias al backend para cerrar un préstamo, calcular las sanciones aplicables y devolver el resultado, simplificando enormemente la lógica requerida en las clases de la interfaz de usuario.

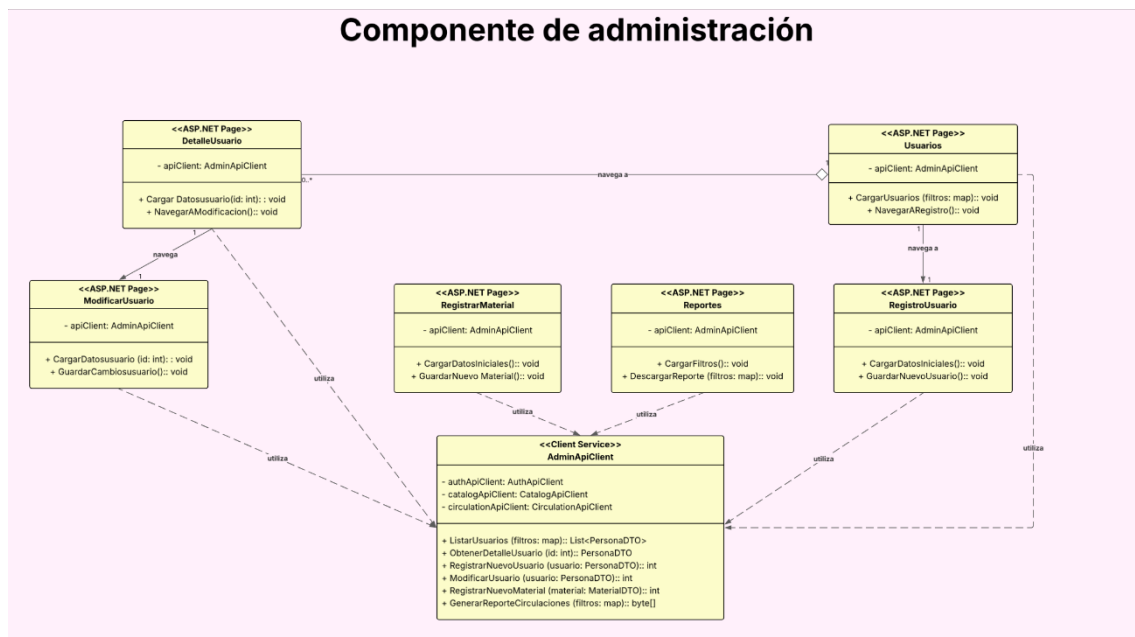
Sustento de decisiones de diseño:

- **Encapsulamiento de la Lógica Transaccional:** El proceso de devolver un libro y generar una sanción es una transacción de negocio. *CirculationApiClient* encapsula esta lógica, asegurando que las llamadas al backend se realicen en el orden correcto y manejando los datos de ida y vuelta (como la lista de sanciones propuestas).
- **Abstracción de Roles:** Este componente está diseñado específicamente para el rol de Bibliotecario. La fachada (*ApiClient*) expone solo las operaciones que este rol necesita, proporcionando una API de cliente segura y enfocada en su tarea.
- **Reutilización y Consistencia:** Al centralizar la lógica en el *CirculationApiClient*, se asegura que todas las páginas que gestionan circulaciones (listado, detalle, registro) lo hagan de manera consistente, reutilizando los mismos métodos para comunicarse con el backend.

Escenario #1: Bibliotecario registra la devolución de un libro con retraso

Descripción: Un bibliotecario busca un préstamo vencido en *Circulaciones* y navega a su *DetalleCirculacion*. En esta página, selecciona la acción "Registrar Devolución" y hace clic en "Procesar". El código de la página invoca al método *ResolverCirculacion* del *CirculationApiClient*. Este cliente envía la solicitud al backend, el cual cierra la circulación, calcula que hubo un retraso y devuelve una propuesta de sanción. El *ApiClient* recibe esta propuesta y la pasa a la página de detalle, que muestra un modal de confirmación. Una vez que el bibliotecario confirma, la página llama al método *RegistrarSanciones* del *ApiClient*, que envía la sanción final al backend para su almacenamiento.

3.4.19 Vista de Código del Componente de Administración - Aplicación Web



Nota. Elaboración propia. Para verlo más a detalle, ingresar a:

https://drive.google.com/file/d/1Y_E3OzE27_I6uhw0NcvfRzVnZifrDG8F/view?usp=sharing

El **AdminApiClient** es una fachada de alto nivel que consolida las funcionalidades administrativas dispersas en varios servicios del backend. Las páginas de administración (**PaginaGestionUsuarios**, **PaginaRegistroMaterial**, **PaginaReportes**) necesitan interactuar con la lógica de usuarios, catálogo y circulaciones. En lugar de que estas páginas dependan de tres **ApiClient**s diferentes (**AuthApiClient**, **CatalogApiClient**, **CirculationApiClient**), dependen de un único **AdminApiClient**. Esta fachada interna delega las llamadas a los **ApiClient**s correspondientes, proveyendo una interfaz cohesiva y simplificada para todas las tareas de gestión, y desacoplando el componente de administración de la estructura interna de los otros clientes de API.

Sustento de decisiones de diseño:

- **Principio de Mínimo Conocimiento (Ley de Demeter):** Las páginas de administración no necesitan saber qué **ApiClient** se encarga de qué. Solo conocen la fachada **AdminApiClient**, que les provee todos los servicios que necesitan. Esto reduce las dependencias y simplifica el código de las páginas.
- **Cohesión Funcional:** Agrupa todas las funcionalidades administrativas en un único punto de entrada en el cliente. Esto hace que el propósito del "Componente de Administración" sea mucho más claro y su código más organizado.
- **Control de Acceso Centralizado:** Al tener una única fachada para las funciones administrativas, se podría implementar en el futuro una capa de seguridad o

logging adicional en el AdminApiClient de forma centralizada, sin tener que modificar múltiples clases.

Escenario #1: Un bibliotecario registra un nuevo material en el sistema

Descripción: El bibliotecario navega a la PaginaRegistroMaterial dentro del módulo de administración. Completa el formulario con los datos del nuevo libro, incluyendo título, autor y la cantidad de ejemplares. Al hacer clic en "Guardar", el código de la página invoca al método RegistrarNuevoMaterial del AdminApiClient. Esta fachada, a su vez, delega la llamada al método correspondiente en el CatalogApiClient, que es el especialista en la gestión del catálogo. El CatalogApiClient envía la solicitud SOAP al backend, que persiste el nuevo material y sus ejemplares en la base de datos y devuelve una confirmación de éxito. El AdminApiClient recibe esta confirmación y la pasa a la página, que muestra un mensaje de "Material registrado exitosamente".

4.4.X Vista de Patrón de Diseño X

<< Diagrama de Clases de Diseño >>

Sustento de decisiones de diseño:

4.4.Y Vista de Persistencia Y

<< Diagrama de Clases de Diseño >>

Sustento de decisiones de diseño:

4.4.Z Vista de Interacción con la Base de Datos Z

<< Diagrama de Clases de Diseño >>

Sustento de decisiones de diseño:

4 PLAN DE PRUEBAS

4.1 CONTEXTO DE LAS PRUEBAS

4.1.1 Uso del Sistema

<Similar a introducción. Enfocarse en lo que se va a probar>

4.1.2 Elementos a Probar

<Si se van a probar todos los componentes o sólo algunos, que no se va a probar. Y porqué. >

4.1.3 Alcance de las Pruebas

<Si se revisará sólo caja negra, conexión con otros sistemas, revisión de desempeño, etc.>

4.1.4 Asunciones

<Listar las Asunciones que afectarían el desarrollo de las pruebas >

4.1.5 Restricciones

<Listar las Restricciones que afectarían el desarrollo de las pruebas >

4.2 COMUNICACIÓN DEL PLAN

ID	Interesados	Forma de Comunicación	Periodicidad
			Semanal/Semestral/Al terminar cada prueba

4.3 ENFOQUE DE PRUEBAS

<Seleccionar el o los enfoques que se usarán al planificar las pruebas. Considerar los 7 descritos en clase y sustentar >

4.4 TIPOS DE PRUEBAS

< Listar 3 tipos de Pruebas y el sustento de su uso >

4.5 RIESGOS EN EL DESARROLLO DE LAS PRUEBAS.

ID	Riesgo	Sustento

4.6 CRONOGRAMA.

Fecha	Actividad	Responsable/Rol

4.7 CASOS DE PRUEBAS

4.7.1 Pantalla 1 a probar.

<< Pantalla>>

4.7.1.1 Tablas de Casos de Equivalencia.

4.7.1.2 Especificación de Caso de Prueba Válido

4.7.1.3 Especificación de Caso de Prueba No Válido.

4.7.2 Pantalla 2 a probar.

<< Pantalla >>

4.7.2.1 Tablas de Casos de Equivalencia.

4.7.2.2 Especificación de Caso de Prueba Válido

Especificación de Caso de Prueba No Válido.